

Claude Code Implementation Plan

Complete Guide to Building RAG Recruiting System with Claude Code

Overview: Using Claude Code Effectively

Claude Code is a command-line tool that allows Claude to write, run, and iterate on code directly in your development environment. Here's how to leverage it for this project.

What Claude Code Can Do:

- Create and modify files across your project
- Run commands and see outputs
- Install dependencies
- Create database migrations
- Test code in real-time
- Debug issues iteratively
- Set up configurations

Before Starting:

1. Install Claude Code CLI
 2. Navigate to your project directory
 3. Ensure Docker is running
 4. Have the project specification document available
-

Phase-by-Phase Implementation Strategy

PHASE 1: Foundation Setup (Days 1-3)

Session 1: Project Initialization

Your Prompt to Claude Code:

I'm building a RAG-based recruiting system. Here's the complete specification:

[Paste the Project Specification document]

Phase 1 Goal: Set up the complete project structure, Docker environment, and database.

Tasks for this session:

1. Create the full project directory structure as specified
2. Set up Docker Compose with PostgreSQL, Redis, and MinIO
3. Initialize FastAPI backend with basic configuration
4. Initialize React + TypeScript frontend with Vite
5. Create .env.example files for both backend and frontend
6. Create requirements.txt with all necessary Python packages
7. Create package.json with all necessary npm packages
8. Set up basic README.md with setup instructions

PostgreSQL is already running in Docker at localhost:5432.

Use these credentials: user=user, password=password, db=recruiting_db

Start with creating the project structure.

What Claude Code Will Do:

- Create entire folder structure
- Generate docker-compose.yml
- Create backend/requirements.txt with dependencies
- Create frontend/package.json
- Set up .env.example files
- Create initial configuration files

Your Follow-up:

Now let's initialize both applications:

1. Set up FastAPI with basic main.py and config.py
2. Set up React with Vite, TypeScript, and basic routing
3. Install all dependencies in both projects
4. Verify both can run successfully

Session 2: Database Models and Migrations

Your Prompt:

Now let's create the complete database schema with SQLAlchemy and Alembic.

Create all 27 database tables as specified in the database schema section:

1. Candidates module (7 tables)
2. Jobs module (5 tables)
3. Social integrations module (9 tables)
4. Vector & analytics module (6 tables)

For each model:

- Use SQLAlchemy ORM with proper relationships
- Include all fields with correct types
- Add indexes on frequently queried fields
- Include timestamps (created_at, updated_at)
- Add proper foreign key constraints

Then:

1. Initialize Alembic
2. Create initial migration
3. Run migration against the PostgreSQL database
4. Verify tables are created

Start with the candidates module models.

What Claude Code Will Do:

- Create all model files in app/models/
- Set up proper relationships and constraints
- Initialize Alembic
- Generate and run migrations
- Verify database connection

Your Follow-up:

Create Pydantic schemas for all models to handle API request/response validation.
Organize them in app/schemas/ directory.

Session 3: Basic CRUD APIs

Your Prompt:

Now let's implement basic CRUD operations for candidates and jobs.

Create:

1. API routers in `app/api/v1/candidates.py` and `app/api/v1/jobs.py`
2. Endpoints for:
 - Create candidate/job
 - List candidates/jobs (with pagination)
 - Get single candidate/job by ID
 - Update candidate/job
 - Delete candidate/job
3. Use proper:
 - Async/await patterns
 - Pydantic schema validation
 - Error handling with HTTP exceptions
 - Database session management
4. Register routers in `main.py`
5. Test each endpoint using the FastAPI docs UI

Start with candidates CRUD.

Session 4: Frontend Foundation

Your Prompt:

Let's set up the React frontend foundation:

1. Create the base layout structure:

- Header component with navigation
- Sidebar component
- Main content area
- Dashboard page

2. Set up routing with React Router:

- /candidates
- /jobs
- /applications
- /dashboard

3. Create API service layer:

- Axios instance with base configuration
- candidateService.ts with all CRUD methods
- jobService.ts with all CRUD methods
- Proper error handling

4. Set up Redux Toolkit or Zustand:

- Store configuration
- Candidate slice
- Job slice

5. Install and configure Material-UI or shadcn/ui

6. Create basic pages:

- Candidates list page (table view)
- Jobs list page
- Dashboard with basic stats

Test that frontend can communicate with backend API.

PHASE 2: Document Processing (Days 4-5)

Session 5: File Upload and Storage

Your Prompt:

Implement resume file upload and storage:

1. Set up MinIO client in backend
2. Create file upload endpoint:
 - POST /api/v1/candidates/{id}/resumes
 - Accept PDF and Word files
 - Validate file size and type
 - Store file in MinIO
 - Save metadata to candidate_resumes table
3. Create file download endpoint:
 - GET /api/v1/resumes/{id}/download
4. Frontend resume upload component:
 - Drag-and-drop file upload
 - File preview
 - Upload progress indicator
 - Multiple file support

Test file upload end-to-end.

Session 6: Document Extraction with Dockling

Your Prompt:

Integrate IBM Granite Dockling for document extraction:

1. Install dockling library (or alternative if unavailable)
2. Create DocumentProcessorService in app/services/document_processor.py:
 - Extract text from PDF files
 - Extract text from Word files
 - Parse structured sections (experience, education, skills)
 - Handle both well-formatted and messy resumes
 - Return structured data
3. Create parsing endpoint:
 - POST /api/v1/resumes/{id}/parse
 - Trigger document processing
 - Store parsed data in candidate_resumes.parsed_data (JSONB)
 - Extract and save to candidate_skills, candidate_experience, candidate_education
4. Make parsing asynchronous with Celery:
 - Create Celery task
 - Return task ID immediately
 - Provide status check endpoint

Test with various resume formats.

Session 7: Semantic Chunking

Your Prompt:

Implement semantic chunking for resumes:

1. Install LangChain and necessary dependencies
2. Create ChunkingService in app/services/chunking_service.py:
 - Implement semantic chunking strategy
 - Implement recursive character splitting as fallback
 - Preserve section boundaries (skills, experience, education)
 - Generate chunks with metadata (chunk_type, chunk_order)
3. Store chunks in resume_chunks table:
 - Link to resume_id and candidate_id
 - Include chunk metadata
 - Maintain order
4. Create chunking endpoint:
 - POST /api/v1/resumes/{id}/chunk
 - Trigger chunking after parsing
 - Store all chunks

Test chunking with both structured and unstructured resumes.

Show me chunk examples for a sample resume.

PHASE 3: Embeddings & Vector Store (Days 6-8)

Session 8: Embedding Service Setup

Your Prompt:

Set up the embedding service with Sentence Transformers:

1. Download and load the embedding model:
 - Use sentence-transformers library
 - Model: all-MiniLM-L6-v2 (download from HuggingFace)
 - Cache the model locally
2. Create EmbeddingService in app/services/embedding_service.py:
 - Load model on initialization
 - embed_text(text: str) -> ndarray
 - embed_batch(texts: List[str]) -> ndarray
 - Proper error handling
 - GPU support if available (check device)
3. Create endpoint to generate embeddings:
 - POST /api/v1/resumes/{id}/embed
 - Get all chunks for resume
 - Generate embeddings for each chunk
 - Return embedding info

Test embedding generation for sample texts.

Show me embedding dimensions and sample output.

Session 9: FAISS Vector Store

Your Prompt:

Set up FAISS vector database:

1. Install faiss-cpu (or faiss-gpu if GPU available)
2. Create VectorService in app/services/vector_service.py:
 - Initialize FAISS index (IndexFlatL2 for start)
 - add_vectors(embeddings, metadata) - add to index
 - search(query_vector, k=100) - similarity search
 - save_index() - persist to disk
 - load_index() - load from disk
 - get_index_stats() - index information
3. Metadata management:
 - Map FAISS index position to chunk_id
 - Store mapping in separate JSON or DB table
 - Enable filtering by candidate_id
4. Integration:
 - After embedding generation, automatically add to FAISS
 - Create endpoint: POST /api/v1/vector/add-resume/{resume_id}
5. Test:
 - Add sample embeddings
 - Perform search query
 - Verify results

Show me index statistics after adding test data.

Session 10: Batch Processing

Your Prompt:

Implement batch processing for embedding existing resumes:

1. Set up Celery worker:
 - Configure Celery with Redis broker
 - Create worker configuration
 - Set up task routing
2. Create batch processing tasks in app/workers/embedding_tasks.py:
 - batch_embed_candidates(candidate_ids: List[UUID])
 - Process in chunks of 100 candidates
 - Update progress in Redis
 - Handle failures gracefully
 - Retry failed embeddings
3. Create batch control endpoints:
 - POST /api/v1/admin/batch-embed - start batch processing
 - GET /api/v1/admin/batch-status/{task_id} - check status
 - POST /api/v1/admin/batch-cancel/{task_id} - cancel
4. Frontend monitoring:
 - Create batch processing dashboard
 - Show progress bar
 - Display success/failure counts
 - Show estimated completion time
5. Test:
 - Create 100 test candidates with resumes
 - Trigger batch processing
 - Monitor progress

Start Celery worker and test batch processing.

PHASE 4: Search & Matching (Days 9-11)

Session 11: Hybrid Search Implementation

Your Prompt:

Implement hybrid search combining PostgreSQL filtering and FAISS semantic search:

1. Create SearchService in app/services/search_service.py:

- structured_filter(filters: dict) -> List[candidate_id]
 - * Filter by location, experience years, salary range
 - * Filter by skills (exact match)
 - * Return candidate IDs
- semantic_search(query: str, candidate_pool: List[UUID]) -> List[Match]
 - * Embed query text
 - * Search FAISS with filter on candidate_pool
 - * Return chunks with scores
 - * Aggregate by candidate_id
- hybrid_search(query: str, filters: dict) -> List[Candidate]
 - * Combine both approaches
 - * First filter, then semantic search
 - * Return ranked candidates with scores

2. Create search endpoints:

- POST /api/v1/search/candidates
- Body: {query, filters: {location, experience_min, experience_max, skills}}
- Return paginated results

3. Test:

- Search for "Python developer with ML experience"
- Apply filters: location=California, experience_min=3
- Verify results are relevant

Show me search results with scores.

Session 12: Reranking Service

Your Prompt:

Add reranking for improved precision:

1. Install cross-encoder model
2. Create RerankingService in app/services/reranking_service.py:
 - Load cross-encoder model: cross-encoder/ms-marco-MiniLM-L-12-v2
 - rerank(query: str, documents: List[str], top_k: int) -> List[RankedDoc]
 - Score each document against query
 - Return top-k sorted by score
3. Integrate with search:
 - After FAISS returns top 100
 - Rerank to get top 20
 - Add reranked_score to results
4. Create A/B test endpoint:
 - Compare results with and without reranking
 - Show score differences

Test reranking and show improvement in result quality.

Session 13: Job-Candidate Matching

Your Prompt:

Implement intelligent job-candidate matching:

1. Create MatchingService in app/services/matching_service.py:

- match_candidates_to_job(job_id: UUID) -> List[Match]
 - * Get job details
 - * Extract requirements (skills, experience, location)
 - * Use hybrid search with job description as query
 - * Calculate multi-factor match score:
 - Semantic similarity (40%)
 - Skills match percentage (30%)
 - Experience match (20%)
 - Location match (10%)
 - * Generate match explanation
- calculate_skill_match(job_skills, candidate_skills) -> float
- calculate_experience_match(required, actual) -> bool
- generate_match_explanation(scores) -> str

2. Save recommendations:

- Store in job_recommendations table
- Include all score components
- Add explanation

3. Create endpoints:

- POST /api/v1/jobs/{id}/match
- GET /api/v1/jobs/{id}/recommendations
- GET /api/v1/candidates/{id}/recommendations

4. Frontend:

- Job matching results page
- Display candidates with match scores
- Show score breakdown (skills, experience, location)
- Visual match indicator (progress bar/gauge)

Test with real job description and verify match quality.

PHASE 5: LLM Generation (Days 12-13)

Session 14: Generation Service Abstraction

Your Prompt:

Create flexible LLM generation service supporting multiple providers:

1. Create abstract base in `app/services/generation/base.py`:
 - Abstract `GenerationProvider` class
 - Methods: `generate()`, `chat()`, `stream()`
2. Implement providers:
 - `app/services/generation/ollama_provider.py`
 - `app/services/generation/openai_provider.py`
 - `app/services/generation/anthropic_provider.py`
3. Create `GenerationService` in `app/services/generation_service.py`:
 - Load provider based on environment config
 - Unified interface for all providers
 - Prompt templates for common tasks
4. Implement use cases:
 - `summarize_resume(resume_text) -> str`
 - `generate_job_description(requirements) -> str`
 - `generate_interview_questions(job, candidate) -> List[str]`
 - `generate_email(type, context) -> str`
5. Create endpoints:
 - POST `/api/v1/generation/summarize-resume`
 - POST `/api/v1/generation/job-description`
 - POST `/api/v1/generation/interview-questions`
6. Frontend integration:
 - AI summary on candidate profile
 - Job description generator
 - Interview question generator

Start with Ollama provider. Test generation quality.

PHASE 6: External Integrations (Days 14-16)

Session 15: LinkedIn Integration

Your Prompt:

Implement LinkedIn profile integration:

1. Set up OAuth 2.0:

- Create OAuth endpoints (authorize, callback)
- Store access tokens securely in database
- Handle token refresh

2. Create LinkedInService in app/services/integration_services/linkedin.py:

- authenticate(code) -> access_token
- get_profile(access_token) -> profile_data
- get_experience(access_token) -> List[experience]
- sync_profile(candidate_id, access_token)
 - * Fetch all profile data
 - * Store in linkedin_profiles table
 - * Update candidate_experience, candidate_skills
 - * Update candidate_education

3. Create endpoints:

- GET /api/v1/integrations/linkedin/authorize
- GET /api/v1/integrations/linkedin/callback
- POST /api/v1/integrations/linkedin/sync/{candidate_id}

4. Frontend:

- "Connect LinkedIn" button
- OAuth popup flow
- Show sync status
- Display LinkedIn data

Test OAuth flow and data sync.

Session 16: GitHub Integration

Your Prompt:

Implement GitHub integration:

1. Set up GitHub OAuth
2. Create GitHubService in app/services/integration_services/github.py:
 - authenticate(code)
 - get_profile(access_token)
 - get_repositories(access_token)
 - get_contributions(access_token)
 - sync_profile(candidate_id, access_token)
 - * Store in github_profiles
 - * Store repos in github_repositories
 - * Extract primary languages
 - * Calculate contribution stats
3. Create endpoints similar to LinkedIn
4. Frontend integration
5. Test with real GitHub account

Session 17: Additional Integrations

Your Prompt:

Implement remaining integrations:

1. Medium/Blog integration:

- RSS feed parsing or API
- Store in articles_blogs table

2. YouTube integration:

- YouTube Data API
- Fetch channel and video data

3. StackOverflow integration:

- StackExchange API
- Fetch profile and reputation

4. Create unified sync service:

- Celery periodic tasks for auto-sync
- Sync all platforms for a candidate
- Store sync logs

5. Frontend:

- Social profiles dashboard
- Connect/disconnect buttons for each platform
- Last sync timestamp
- Sync now button

Test all integrations end-to-end.

PHASE 7: Deduplication (Days 17-18)

Session 18: Deduplication Service

Your Prompt:

Implement candidate deduplication:

1. Create DeduplicationService in app/services/deduplication_service.py:

- detect_duplicates(candidate_id: UUID) -> List[Duplicate]
 - * Method 1: Exact email/phone match
 - * Method 2: Fuzzy name matching (Levenshtein distance)
 - * Method 3: Resume content similarity (use embeddings)
 - * Method 4: Employment history overlap
- calculate_similarity_score(candidate1, candidate2) -> float
- merge_candidates(primary_id, duplicate_id)
 - * Update all foreign keys
 - * Merge skills, experience, education
 - * Keep most recent data
 - * Mark duplicate as merged
 - * Maintain audit trail

2. Create duplicate_candidates entries automatically:

- Run detection when new candidate added
- Run detection after resume upload
- Score duplicates (0-100)

3. Create endpoints:

- GET /api/v1/admin/duplicates - list all duplicates
- POST /api/v1/admin/detect-duplicates/{candidate_id}
- POST /api/v1/admin/merge-candidates
- POST /api/v1/admin/dismiss-duplicate

4. Frontend:

- Duplicate candidates page
- Side-by-side comparison
- Merge interface
- Field selection for merge

Test with intentionally similar candidates.

PHASE 8: Application Workflow (Days 19-20)

Session 19: Application Management

Your Prompt:

Implement complete application workflow:

1. Create ApplicationService:

- submit_application(candidate_id, job_id, resume_id)
- update_status(application_id, new_status, notes)
- get_application_history(application_id)
- get_candidate_applications(candidate_id)
- get_job_applications(job_id)

2. Create endpoints:

- POST /api/v1/applications/
- GET /api/v1/applications/{id}
- PUT /api/v1/applications/{id}/status
- GET /api/v1/applications/{id}/history
- GET /api/v1/candidates/{id}/applications
- GET /api/v1/jobs/{id}/applications

3. Status workflow:

- submitted → under_review → shortlisted → interview_scheduled
- → interview_completed → offer_extended → offer_accepted/rejected
- → rejected (can happen at any stage)
- Track all transitions in application_status_history

4. Frontend:

- Application form
- Application status pipeline (Kanban board)
- Application details view
- Status update interface

Test complete workflow from submission to hire.

Session 20: Interview Scheduling

Your Prompt:

Implement interview scheduling:

1. Create InterviewService:

- schedule_interview(application_id, details)
- update_interview(interview_id, details)
- add_feedback(interview_id, feedback, rating)
- get_upcoming_interviews(recruiter_id)

2. Create endpoints for interview CRUD

3. Frontend:

- Interview scheduler component
- Calendar view
- Feedback form
- Rating system

4. Optional: Integrate with Google Calendar

Test scheduling and feedback collection.

PHASE 9: Analytics & UI Polish (Days 21-22)

Session 21: Analytics Dashboard

Your Prompt:

Create analytics and monitoring:

1. Create AnalyticsService:

- Get application metrics (conversion rates, time-to-hire)
- Get search quality metrics (click-through rate)
- Get system usage stats
- Get match accuracy metrics

2. Create endpoints:

- GET /api/v1/analytics/applications
- GET /api/v1/analytics/search
- GET /api/v1/analytics/system

3. Frontend dashboard:

- Recruiter dashboard (applications, interviews, stats)
- Candidate dashboard (applications, recommendations)
- System dashboard (embeddings, FAISS stats, processing)
- Charts using Recharts or Chart.js

Create comprehensive dashboard with key metrics.

Session 22: UI/UX Polish

Your Prompt:

Polish the frontend:

1. Improve UI/UX:

- Consistent styling across all pages
- Loading states for all async operations
- Error handling and user feedback
- Empty states
- Responsive design (mobile-friendly)

2. Add features:

- Search with autocomplete
- Filters with chips
- Sort options
- Export functionality (CSV, PDF)
- Bulk operations

3. Accessibility:

- ARIA labels
- Keyboard navigation
- Screen reader support

4. Performance:

- Lazy loading
- Code splitting
- Image optimization
- Caching

Review entire application and polish all rough edges.

PHASE 10: Testing & Deployment (Days 23-25)

Session 23: Backend Testing

Your Prompt:

Create comprehensive test suite for backend:

1. Unit tests for services:
 - Test EmbeddingService
 - Test VectorService
 - Test MatchingService
 - Test DeduplicationService
 - Mock external dependencies
2. Integration tests for APIs:
 - Test all CRUD endpoints
 - Test search endpoints
 - Test matching endpoints
 - Use pytest fixtures
3. Test database operations:
 - Test migrations
 - Test relationships
 - Test constraints
4. Run tests:
 - `pytest --cov`
 - Aim for >80% coverage

Create tests and show coverage report.

Session 24: Frontend Testing

Your Prompt:

Create frontend tests:

1. Component tests:
 - Test key components
 - Test user interactions
 - Mock API calls
2. Integration tests:
 - Test complete user flows
 - Test form submissions
 - Test search functionality
3. Run tests and show results.

Session 25: Deployment Preparation

Your Prompt:

Prepare application for deployment:

1. Production Docker setup:
 - Multi-stage Dockerfile for backend
 - Production build for frontend
 - Nginx configuration
 - Environment-specific configs
2. Security:
 - Review CORS settings
 - Add rate limiting
 - Secure environment variables
 - Add API authentication
3. Documentation:
 - Update README with deployment instructions
 - API documentation review
 - Architecture documentation
 - User guide
4. Performance optimization:
 - Database query optimization
 - API response caching
 - FAISS index optimization
 - Frontend bundle optimization
5. Monitoring:
 - Add health check endpoints
 - Set up logging
 - Error tracking

Prepare complete deployment package.

Tips for Working with Claude Code

1. Be Specific and Iterative

Don't ask Claude Code to build everything at once. Break it down:

-  "Create the candidate model with all fields"

- ❌ "Build the entire backend"

2. Provide Context

Always reference the specification document and current phase:

Continuing from Phase 3, Session 9.
We've completed embedding service setup.
Now let's implement FAISS vector store.

3. Test Incrementally

After each component, ask Claude Code to test it:

Test the embedding service:

1. Generate embeddings for sample text
2. Verify dimension is correct
3. Test batch embedding

Show me the output.

4. Request Explanations

Ask Claude Code to explain complex implementations:

Explain how the hybrid search works and why we filter first before semantic search.

5. Handle Errors Together

When errors occur:

I'm getting this error: [paste error]
Debug this and fix the issue.

6. Review and Improve

After implementation:

Review the MatchingService code:

1. Are there any performance issues?
2. Can we improve the algorithm?
3. Is error handling comprehensive?

7. Save Progress

After each major milestone:

Create a checkpoint:

1. Commit all changes
2. Update progress in README
3. List what's completed and what's next

Session Templates

Starting a New Session

I'm continuing the RAG Recruiting System project.

Current Status: [Phase X, Session Y]

Completed: [list completed items]

Next Goal: [specific goal]

Reference: [paste relevant spec section]

Let's start with: [specific first task]

Debugging Session

I encountered an issue with [component].

Error: [paste error]

Context: [what you were doing]

Please:

1. Identify the root cause
2. Fix the issue
3. Add error handling to prevent this
4. Test the fix

Review Session

Review the [component] we built:

1. Code quality and best practices
2. Performance implications
3. Security considerations
4. Test coverage
5. Documentation

Suggest improvements.

Progress Tracking

Create a progress.md file and update after each session:

```
markdown

# RAG Recruiting System - Progress

## Phase 1: Foundation ✅
- [x] Project structure
- [x] Docker setup
- [x] Database models
- [x] Basic APIs
- [x] Frontend foundation

## Phase 2: Document Processing ⚠️
- [x] File upload
- [x] Document extraction
- [ ] Semantic chunking (in progress)

## Phase 3: Embeddings & Vector Store
- [ ] Embedding service
- [ ] FAISS setup
- [ ] Batch processing

...

## Current Session
Phase 2, Session 7: Semantic Chunking
Next: Complete chunking and test with various resumes
```

Success Checklist

After completing all phases, verify:

Functionality

- ☐ Resume upload and parsing works
- ☐ Semantic chunking preserves context
- ☐ Embeddings generated correctly
- ☐ FAISS search returns relevant results
- ☐ Reranking improves precision
- ☐ Job-candidate matching is accurate
- ☐ Deduplication detects duplicates
- ☐ All integrations sync data

- ☐ Application workflow is complete
- ☐ LLM generation works (both on-prem and cloud)

Performance

- ☐ API responses < 500ms
- ☐ Search results < 2 seconds
- ☐ Batch processing handles scale
- ☐ Frontend loads quickly

Quality

- ☐ Test coverage > 80%
- ☐ No critical security issues
- ☐ All APIs documented
- ☐ Code is maintainable

User Experience

- ☐ UI is intuitive
- ☐ Loading states are clear
- ☐ Errors are handled gracefully
- ☐ Mobile-friendly

Ready to Start!

You now have:

1. ☒ Complete project specification
2. ☒ Phase-by-phase implementation plan
3. ☒ Session-by-session guide
4. ☒ Claude Code usage strategies
5. ☒ Templates and examples

Start with Phase 1, Session 1:

Claude Code, I'm building a RAG-based recruiting system.

[Paste project specification]

Phase 1, Session 1: Let's initialize the project structure.

Create all folders and files as specified.

Good luck building your intelligent recruiting system! 🚀

